

Introduction to Computers and Programming in C

Module V **Structures**

Structures

- Arrays are used to store large set of data and manipulate them, but the disadvantage is that all the elements stored in an array are to be of the same data type.
- If we need to use a collection of different data type items, it is not possible using an array.
- When we require using a collection of different data items of different data types, we can use a structure.
- **Structure is a method of packing data of different types. A structure is a convenient method of handling a group of related data items of different data types.**

Structures

General format:

```
struct tag_name
{
    data type member1;
    data type member2;
    ...
    ...
};
```

Structures

Example:

```
struct lib_books
{
    char title[20];
    char author[15];
    int pages;
    float price;
};
```

Structures

- the keyword `struct` declares a structure to holds the details of four fields namely title, author pages and price.
- These are members of the structures.
- Each member may belong to different or same data type.
- The tag name can be used to define objects that have the tag names structure.
- The structure we just declared is not a variable by itself but a template for the structure.

Structures

- We can declare structure variables using the tag name anywhere in the program.
- For example the statement,

```
struct lib_books book1, book2, book3;
```

declares book1,book2,book3 as variables of type struct lib_books
each declaration has four elements of the structure lib_books.

Structures

- The complete structure declaration might look like this

```
struct lib_books
{
    char title[20];
    char author[15];
    int pages;
    float price;
};
```

```
struct lib_books book1, book2, book3;
```

Structures

We can also combine both template declaration and variables declaration in one statement, the declaration

```
struct lib_books  
{  
    char title[20];  
    char author[15];  
    int pages;  
    float price;  
} book1,book2,book3;
```

is valid.

Structures

- book1, book2, book3 are declared as structure variables representing 3 books but does not include a tag name for use in the declaration.
- A structure is usually defining before main along with macro definitions. In such cases the structure assumes global status and all the functions can access the structure.

Giving Values to Members

- The link between a member and a structure variable is established using the member operator ‘.’ Which is known as dot operator or period operator.

For example:

`book1.price` is the variable representing the price of book1 and can be treated like any other ordinary variable. We can use scanf statement to assign values like

```
scanf(“%s”,book1.file);  
scanf(“%d",&book1.pages);
```

```
book1.pages=250;  
book1.price=28.50;
```

Example

```
#include<stdio.h>
struct student
{
    int id_no;
    char name[20];
    char address[20];
    int age;
}newstudent;
```

Example

```
void main()
{
    printf("Enter the student information");
    printf("Now Enter the student id_no");
    scanf("%d",&newstudent.id_no);
    printf("Enter the name of the student");
    scanf("%s",&newstudent.name);
    printf("Enter the address of the student");
    scanf("%s",&newstudent.address);
    printf("Enter the age of the student");
    scanf("%d",&newstudent.age);
    printf("Student information\n");
    printf("student id_number=%d\n",newstudent.id_no);
    printf("student name=%s\n",newstudent.name);
    printf("student Address=%s\n",newstudent.address);
    printf("Age of student=%d\n",newstudent.age);
}
```

Initializing structure

Example:

```
struct student newstudent = { 12345, “kapildev”, “Pes college”,  
    19};
```

This initializes the `id_no` field to 12345,
the `name` field to “kapildev”,
the `address` field to “pes college” and
the `age` field to 19.

Functions and Structure

- We can pass structures as arguments to functions. Unlike array names however, which always point to the start of the array, structure names are not pointers. As a result, when we change structure parameter inside a function, we don't effect its corresponding argument.

Passing structure to elements to functions:

A structure may be passed into a function as individual member or a separate variable.

Functions and Structure

- A program example to display the contents of a structure passing the individual elements to a function is shown below.

```
#include<stdio.h>
struct emp
{
    int emp_id;
    char name[25];
    char department[10];
    float salary;
};
void display(int,char []);
void main()
{
    static struct emp emp1={125,"sampath","operator",7500.00};
    display(emp1.emp_id,emp1.name);
}
/* function to display structure variables*/
void display(int e_no, char e_name[])
{
    printf("%d %s", e_no, e_name);
}
```

Arrays and Structure

It is possible to define a array of structures for example if we are maintaining information of all the students in the college and if 100 students are studying in the college. We need to use an array than single variables. We can define an array of structures as shown in the following example:

```
structure information
{
    int id_no;
    char name[20];
    char address[20];
    char combination[3];
    int age;
}student[100];
```

Arrays and Structure

- An array of structures can be assigned initial values just as any other array can. Remember that each element is a structure that must be assigned corresponding initial values as illustrated below.

```
#include<stdio.h>
void main()
{
    struct info
    {
        int id_no;
        char name[20];
        char address[20];
        int age;
    }std[100];
    int l,n;
    printf("Enter the number of students");
    scanf("%d",&n);
    printf(" Enter Id_no,name address combination age\n");
    for(l=0;l<n;l++)
        scanf ("%d%s%s%d", &std[l].id_no,std[l].name,std [l].address,&std[l].age);
    printf("\n Student information");
    for (l=0;l< n;l++)
        printf("%d %s %s %d\n",std[l].id_no,std[l].name,std[l].address,std[l].age );
}
```

Structure within Structure

- A structure may be defined as a member of another structure. In such structures the declaration of the embedded structure must appear before the declarations of other structures.

```
struct date
```

```
{  
    int day;  
    int month;  
    int year;  
};
```

```
struct student
```

```
{  
    int id_no;  
    char name[20];  
    char address[20];  
    int age;  
    struct date def;  
    struct date doa;  
}oldstudent,newstudent;
```

the sturcture student contains another structure date as its one of its members.

Union

- Unions like structure contain members whose individual data types may differ from one another. However the members that compose a union all share the same storage area within the computer's memory whereas each member within a structure is assigned its own unique storage area. Thus unions are used to observe memory. They are useful for application involving multiple members. Where values need not be assigned to all the members at any one time.

Union

Like structures union can be declared using the keyword union as follows:

```
union item
{
    int m;
    float p;
    char c;
}
code;
```

this declares a variable code of type union item. The union contains three members each with a different data type. However we can use only one of them at a time. This is because if only one location is allocated for union variable irrespective of size. The compiler allocates a piece of storage that is large enough to access a union member we can use the same syntax that we use to access structure members. That is

```
code.m
code.p
code.c
```

are all valid member variables.

```
#include <stdio.h>
union unionJob
{
    //defining a union
    char name[32];
    float salary;
    int workerNo;
} uJob;

struct structJob
{
    char name[32];
    float salary;
    int workerNo;
} sJob;

int main()
{
    printf("size of union = %d bytes", sizeof(uJob));
    printf("\nsize of structure = %d bytes", sizeof(sJob));
    return 0;
}
```

References

- Programming in ANSI C, E. Balagurusamy. McGrawHill
- Let Us C, Yashvant Kanetkar, BPB Publications
- Programming in C, Reema Thareja, Oxford University Press

Thank You